

SGBD



Administrer MariaDB





Installation de MariaDB-Server

- ❑ L'installation du SGBD sur Debian est simple puisque MariaDB-Server est disponible dans les dépôts.
- ❑ Première étape mettre à jour le référenciel des logiciels :
`>sudo apt update`
- ❑ Si besoin, mettre à jour le système
`>sudo apt upgrade`
- ❑ Installer le serveur de BD :
`>sudo apt install mariadb-server`
- ❑ A la fin de l'installation on peut vérifier le statut du server :
`>systemctl status mariadb`
- ❑ Le message "active (running)", alors tout est OK est le SGBD est fonctionnel.



Le compte "root" de MariaDB-Server

- ❑ Changer le mot de passe "root" avec la commande :

```
>mysqladmin -u root password 'NouveauMotDePasse'
```

- ❑ Si message *"Access denied for user 'root'@'localhost' "*, alors, on force avec la commande :

```
>sudo mysql
```

- ❑ Pour modifier le mot de passe de l'utilisateur "root", on peut exécuter la requête après connexion avec la base de données :

```
>SET PASSWORD FOR 'root'@'localhost' =  
PASSWORD('NouveauMotDePasse');
```

- ❑ Pour confirmer la modification et pour l'enregistrer :

```
>flush privileges;
```

- ❑ Pour quitter MariaDB :

```
>quit;
```



Le compte "root" de MariaDB-Server

- ❑ Pour vérifier le mot de passe "root", on se reconnecte avec le nouveau mot de passe :

```
>mysql -u root -p
```

- ❑ La configuration d'un mot de passe "root" pour MySQL/MariaDB est indispensable, différent de celui du système!
- ❑ Pour visualiser les BD déjà présentes sur le serveur :

```
MariaDB [(none)]> show databases;
```

```
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema     |
+-----+
3 rows in set (0.001 sec)
MariaDB [(none)]>
```



Script de sécurité

- ❑ Un script est utilisé pour changer le mot de passe "root" et configurer certains aspects de sécurité de MySQL avec la commande :

```
>mysql_secure_installation
```

- ❑ La suite est un questionnaire avec réponses "Y" ou "N" :
 - `Set root password` – Définir le mot de passe root (déjà défini, on répond N)
 - `Socket authentication` - Définir le mot de passe en format unic-socket (N)
 - `Remove anonymous users` – Supprimer les utilisateurs anonymes (Y)
 - `Disallow root login remotly` - Supprimer l'accès distant de root (Y)
 - `Remove test database and access to it` – Supprimer la base de données de test et y accéder (Y)
 - `Reload privilege tables now` – Recharger les tables de privilèges maintenant (Y)



Gestion BD et compte MariaDB

- ❑ Créer une BD avec un utilisateur sur le SGBD MariaDB
 >create database <NomdeLaBD>; # Ex: BD "etudiant"
- ❑ Un utilisateur est associé à la BD pour faire la maintenance
 >CREATE USER 'utilisateur'@'ip' IDENTIFIED BY
 'motdepasse';
- ❑ Pour l'utilisateur définir les privilèges d'accès aux données
 >GRANT ALL PRIVILEGES ON NomBaseDeDonnees.* TO
 'utilisateur'@'ip' WITH GRANT OPTION;
- ❑ Offrir à l'utilisateur des privilèges sur les tables de cette BD :
 >GRANT ALL PRIVILEGES ON <NomBD>.* TO
 '<utilisateur>'@'localhost' WITH GRANT OPTION;
- ❑ Confirmer les modifications :
 >FLUSH PRIVILEGES;



Gestion BD et compte MariaDB

- ❑ Créer des tables dans la BD MariaDB
- ❑ Pour avoir accès aux tables de la BD il faut sélectionner la BD :
`>USE <nom_BD>;`
- ❑ Pour créer une table dans la BD activée :
`>CREATE [TEMPORARY] [IF NOT EXISTS]
TABLE <nom_table> (<nom_attribute> TYPE_ATTRIBUT
[OPTIONS]);`
- ❑ Afficher la liste des tables dans la BD :
- ❑ `MariaDB [<nom_BD>]> show tables;`
- ❑ Afficher la structure de la table créée :
- ❑ `MariaDB [<nom_DB>]> describe <nom_table>;`



Gestion BD et compte MariaDB

- ❑ Créer une BD et les tables à partir d'une archive
- ❑ Pour restaurer/créer une base de données sur le serveur MariaDB à partir/vers une archive on utilise la commande :

```
>mysql -u root -p nom_BD < nom_fichier.sql
```

- ❑ Le `nom_fichier.sql` étant une sauvegarde d'une BD avec les tables qui la constituent.
- ❑ Une autre possibilité d'intégrer les données d'une archive ".sql" dans une BD :
 - ❑ On se connecte au MariaDB avec les identifiants "root". Sur la ligne de commande on choisit la BD à traiter :

```
>use nom_BD;
```
 - ❑ L'insertion des données se fait avec la commande :

```
>source chemin_vers_le_fichier.sql
```




Gestion BD et compte MariaDB

- ❑ Pour sauvegarder une BD avec ses tables, on peut utiliser `mysqldump` avec les paramètres nom de la BD et le nom de l'archive SQL :

```
>mysqldump -u root -p nom_BD > nom_archive.sql
```

- ❑ Il est possible de compresser le fichier de sauvegarde SQL avec `gzip` :

```
>mysqldump -u root -p nom_BD | gzip > nom_archive.sql.gz
```



Gérer les privilèges des utilisateurs

- ☐ A la connexion au SGBD MariaDB, on s'identifie avec un nom d'utilisateur.
- ☐ Chaque utilisateur possède une série de privilèges, relatifs aux données stockées sur le serveur :
 - ☐ de sélectionner les données,
 - ☐ de les modifier,
 - ☐ de créer des objets,
 - ☐ etc.
- ☐ Ces privilèges peuvent exister à plusieurs niveaux :
 - ☐ base de données,
 - ☐ tables,
 - ☐ colonnes,
 - ☐ procédures
 - ☐ etc.



Gérer les privilèges des utilisateurs

- ❑ Les informations concernant les utilisateurs sont stockées dans la base de données "**mysql**".
- ❑ Les utilisateurs sont enregistrés dans la table "**user**", avec leurs privilèges globaux.
- ❑ La base "**mysql**" possède 4 tables permettant de stocker les privilèges des utilisateurs à différents niveaux.
 - ❑ "**db**" : privilèges au niveau des bases de données.
 - ❑ "**tables_priv**" : privilèges au niveau des tables.
 - ❑ "**columns_priv**" : privilèges au niveau des colonnes.
 - ❑ "**proc_priv**" : privilèges au niveau des routines.



Gérer les privilèges des utilisateurs

- ❑ La requête à utiliser pour créer un utilisateur :

```
>CREATE USER 'login'@'hote' [IDENTIFIED BY 'mot_de_passe'];
```

- ❑ La requête à utiliser pour supprimer un utilisateur :

```
>DROP USER 'login'@'hote';
```

- ❑ L'utilisateur est défini avec deux éléments :

- ❑ login (l'identifiant de l'utilisateur), ex. : "tom"

- ❑ hôte de connexion (le point d'accès à BD), ex. : "iut.rt.net"

- ❑ L'hôte est l'adresse à partir de laquelle l'utilisateur se connecte. Si l'utilisateur se connecte à partir de la machine locale (là où est le serveur MariaDB), on peut utiliser 'localhost'. Sinon, on utilise en général une adresse IP ou un nom de domaine. Exemples :

```
>CREATE USER 'toto'@'localhost' IDENTIFIED BY 'tonton';
```

```
>CREATE USER 'pifou'@'194.28.12.4' IDENTIFIED BY 'pasglop';
```

```
>CREATE USER 'jerry'@'iut.rt.net' IDENTIFIED BY 'tomlechat';
```



Gérer les privilèges des utilisateurs

- ❑ Il est possible de se connecter à partir de plusieurs hôtes différents en utilisant le joker "%". Ex :

- ❑ L'utilisateur "jerry" peut se connecter à partir de n'importe quel hôte du domaine "rt.net"

```
>CREATE USER 'jerry'@'%.rt.net' IDENTIFIED BY 'lasouris';
```

- ❑ "tom" peut se connecter à partir de n'importe quel hôte dont l'adresse IP commence par 194.28.1.

```
>CREATE USER 'tom'@'194.28.1.%' IDENTIFIED BY 'lechat';
```

- ❑ "paul" est autorisé de se connecter à partir de n'importe quel hôte

```
>CREATE USER 'paul'@'%' IDENTIFIED BY 'looking4you';
```



Gérer les privilèges des utilisateurs

- ❑ La création d'un utilisateur ne lui donne pas automatiquement les droits d'accès et de traitement des données dans la BD. Il faut lui préciser ses privilèges. Pour accorder tous les droits sur une table à 'jean'@'localhost'.

```
>GRANT ALL ON <BD>.<table> TO 'jean'@'localhost';
```

- ❑ Il est possible de préciser les droits d'actions sur les données pour un utilisateur. Ex :

- ❑ Pour l'utilisateur 'jeanne'@'localhost', on définit les privilèges SELECT, INSERT et DELETE sur une table "<BD>.<table>", et UPDATE sur les colonnes "<attribut1>", "<attribut2>".

```
>GRANT SELECT, UPDATE (<attribut1>, <attribut2>),  
DELETE, INSERT ON <BD>.<table> TO 'jeanne'@'localhost'  
IDENTIFIED BY 'exemples2023';
```



Commandes MariaDB

- ❑ **CREATE** – Permet aux utilisateurs de créer des bases de données/tableaux
- ❑ **SELECT** – Permet aux utilisateurs de récupérer des données
- ❑ **INSERT** – Permet aux utilisateurs d'ajouter de nouvelles entrées dans les tableaux
- ❑ **UPDATE** – Permet aux utilisateurs de modifier les entrées existantes dans les tableaux
- ❑ **DELETE** – Permet aux utilisateurs de supprimer les entrées dans le tableau
- ❑ **DROP** – Permet aux utilisateurs de supprimer des bases de données/tableaux entiers

SGBD



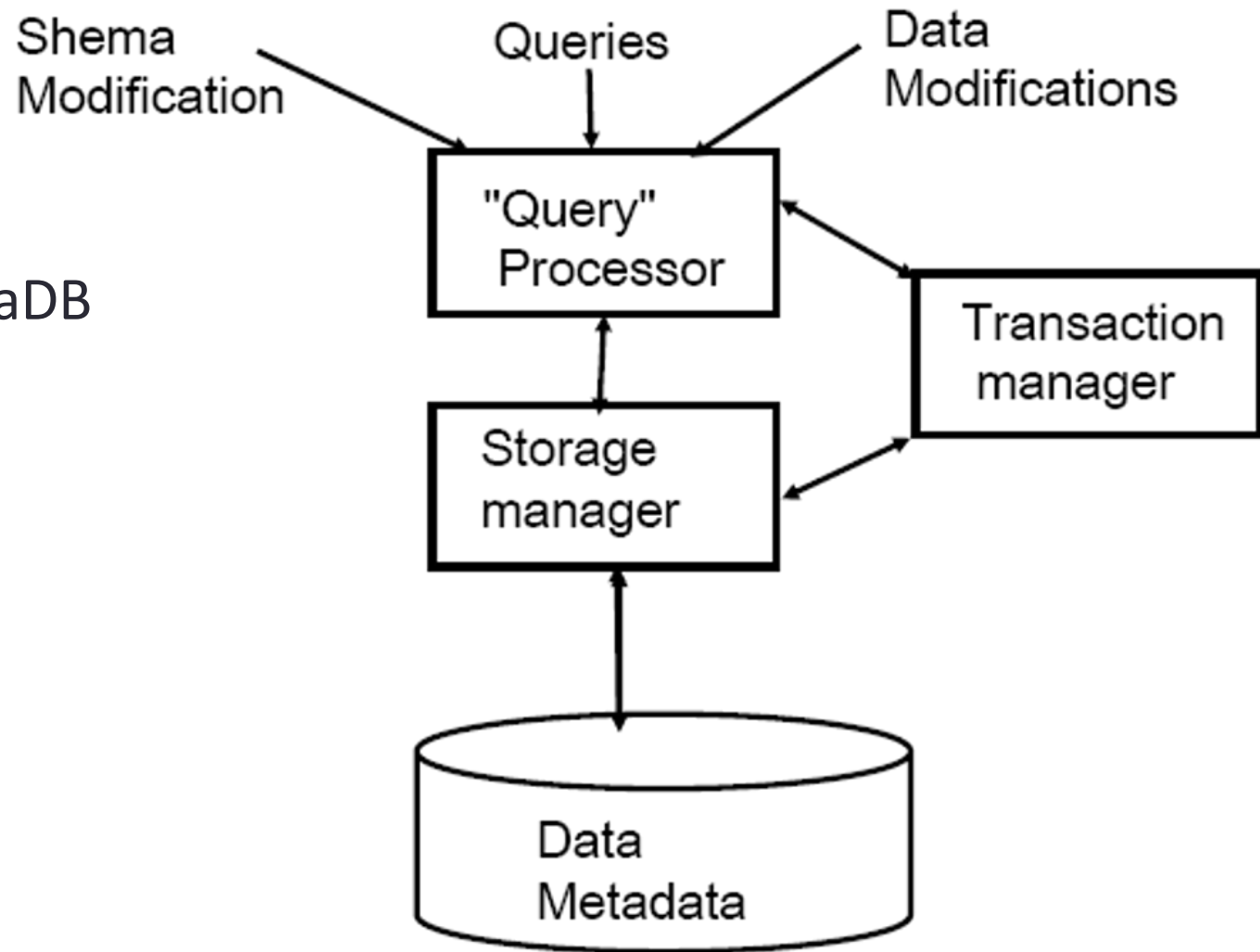
Les Bases de Données





SGBDs & architecture

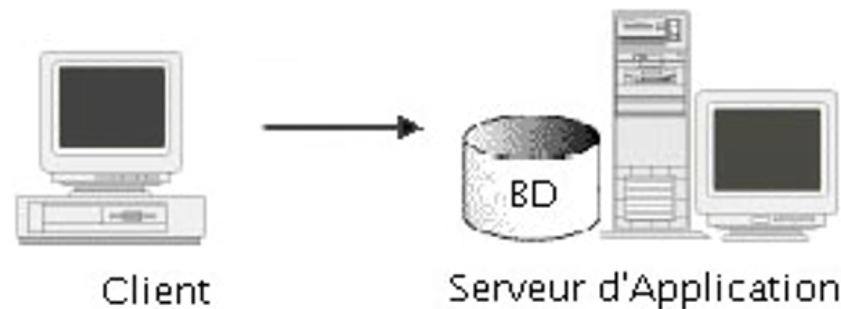
- ☐ MsAccess
- ☐ SQL Server
- ☐ Oracle
- ☐ MySQL & MariaDB
- ☐ DB2
- ☐ PostgreSQL
- ☐ MongoDB
- ☐ SQLite
- ☐ Neo4j
- ☐ Etc.





SGBDs & architecture

- ❑ Un SGBD est :
 - ❑ Un outil pour vérifier, modifier et rechercher efficacement des données dans un grand ensemble
 - ❑ Une interface entre les applications "utilisateur" et la mémoire externe.

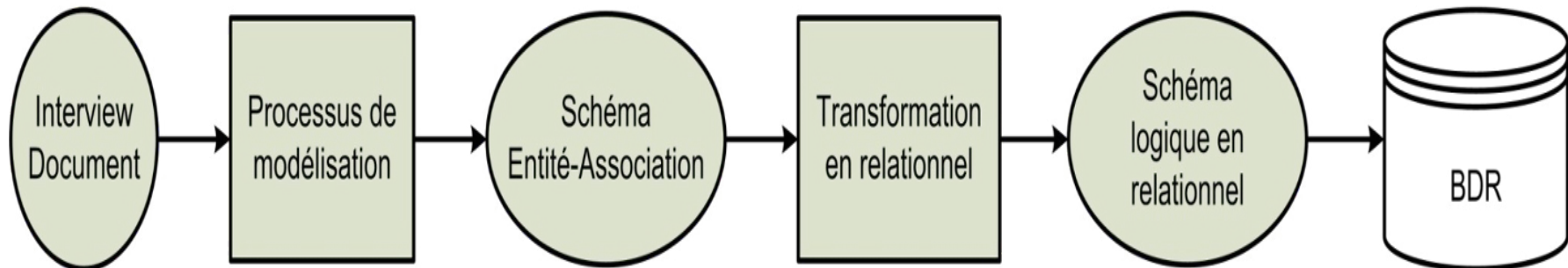


- ❑ Un SGBD comprend :
 - ❑ Un système de gestion de fichiers (gestion du stockage physique).
 - ❑ Un système interne (placement et accès aux données)
 - ❑ Un système externe (langage de requête élaboré).



Modèle entité association (EA)

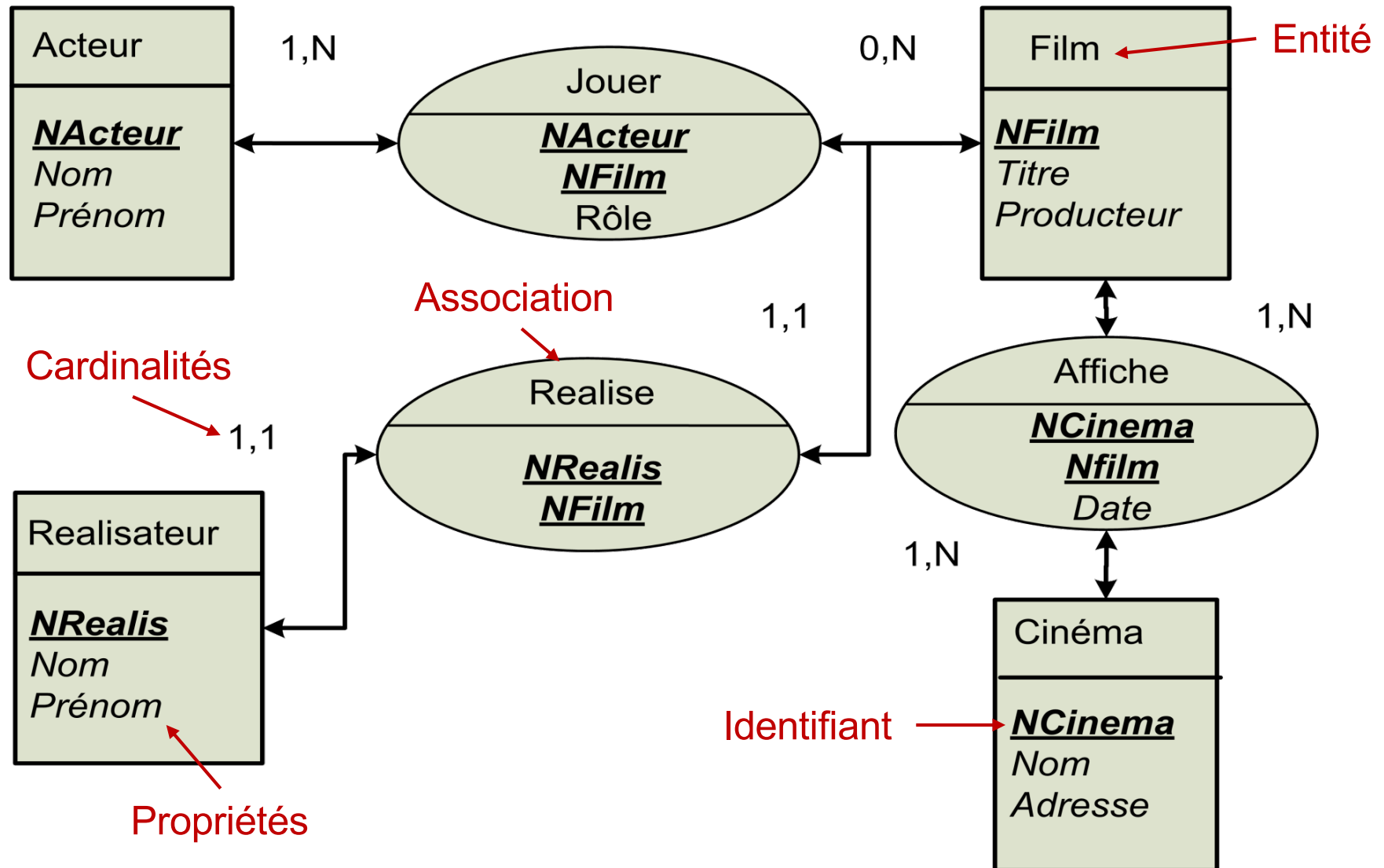
- ❑ Démarche de construction d'une BD :
 - ❑ Construire un schéma conceptuel, modélisé sous forme d'entités et d'associations;
 - ❑ Transformer le schéma E/A en schéma relationnel;
 - ❑ Les étapes de la conception :





Modèle entité association (EA)

❑ Schéma global d'une BD :



PYTHON



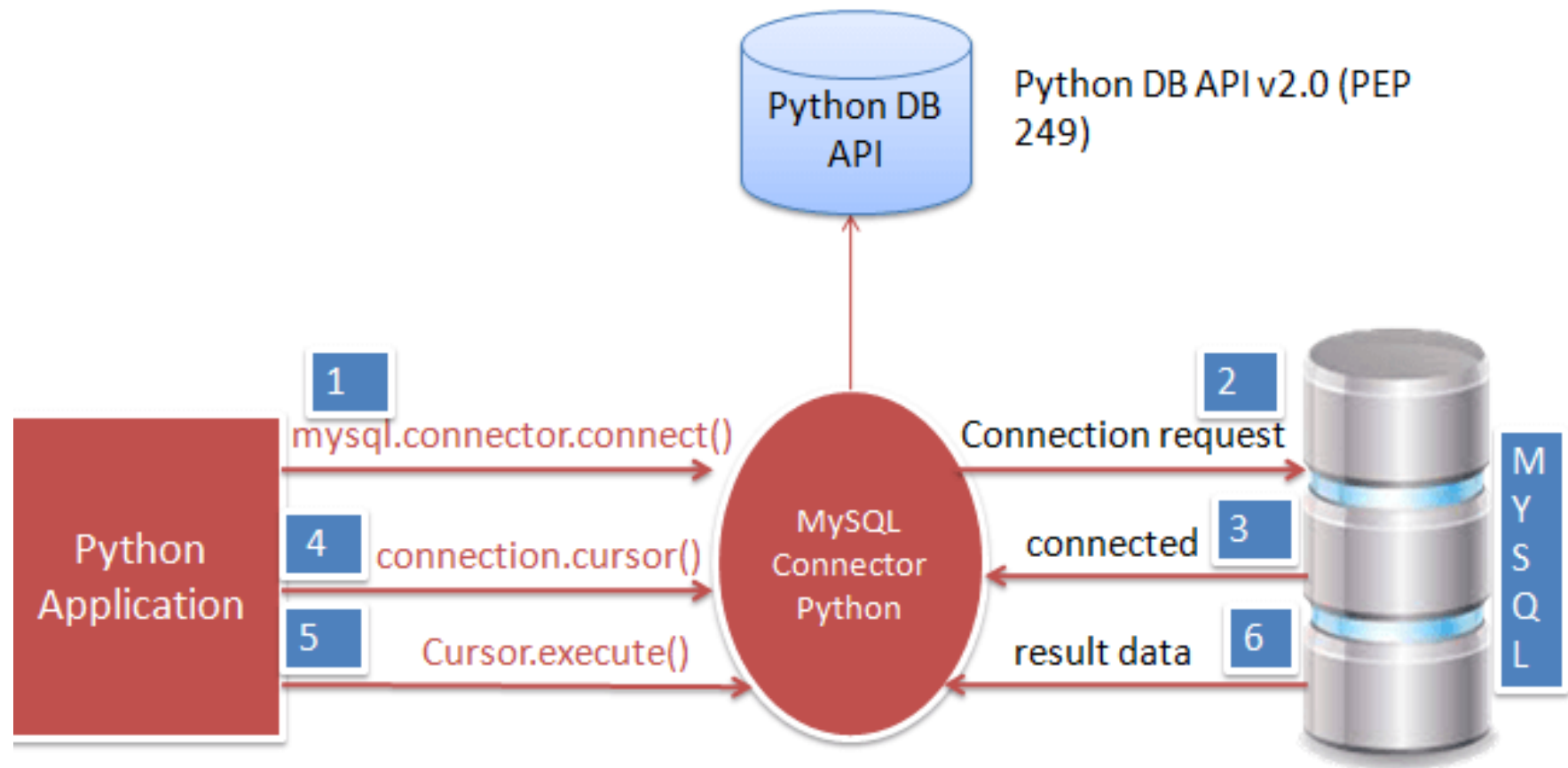
Python & MariaDB





CGI : Python : mysql.connector

- Le schéma d'accès à une base de données MariaDB (MySQL) par un script Python



Installation du connecteur MySQL



- ❑ Pour se connecter à une base de données MySQL, il faut installer le connecteur MySQL qui est un paquet nommé `mysql-connector-python` disponible sur PyPI.

```
>python -m pip install mysql-connector-python==8.0.29
```

- Le module `mysql.connector` fournit la méthode `connect` qui retourne l'objet qui référence la connexion vers la BD.
- On fournit les paramètres "`host`", "`user`" et "`password`" pour donner l'adresse du SGBDR, le login et le mot de passe de connexion.
- On peut également fournir le paramètre `database` pour indiquer la base de données à utiliser.

Connexion à la base de données



- ❑ La connexion à la base de données se comporte comme un gestionnaire de ressource.

```
import mysql.connector

try:
    db = mysql.connector.connect(
        host = "localhost",
        user = "login",
        password = "mot_de_passe",
        database = "database_python"
    )
except mysql.connector.Error as e:
    print("Exception : ", e)
else:
    print("Connexion réussie")
    db.close()
```


Connexion à la base de données



- ❑ Avec le gestionnaire de ressource on peut employer la syntaxe `with` pour s'assurer de la fermeture de la connexion.
- ❑ On peut employer l'opérateur `**` pour faire `unpack` du dictionnaire pour les paramètres de connexion.
- ❑ L'interaction avec la base de données se fait à travers un `curseur` qui permet à la fois d'envoyer des requêtes et de consulter les résultats.
- ❑ Pour créer l'objet `curseur`, on appelle la méthode `cursor()` en référençant la connexion.
- ❑ L'objet `curseur` est aussi un gestionnaire de ressource, alors on peut également employer la syntaxe `with`

Connexion à la base de données



- ❑ Avec le gestionnaire curseur :

```
import mysql.connector
```

```
try:
```

```
    connection_params = {  
        'host': "localhost",  
        'user': "login",  
        'password': "mot_de_passe",  
        'database': "database_python"  
    }
```

```
except mysql.connector.Error as err:
```

```
    print("Connexion échouée: {}".format(err))
```

```
else:
```

```
    with mysql.connector.connect(**connection_params) as db :  
        with db.cursor() as c:  
            print("Connexion réussie")  
        db.close()
```

```
import mysql.connector  
  
try:  
    connection_params = {  
        'host' : "192.168.1.41",  
        'user' : "jerry",  
        'password' : "disney",  
        'database' : "etudiant"  
    }  
except mysql.connector.Error as e:  
    print("Exception : ", e)  
else:  
    with mysql.connector.connect(**connection_params) as db :  
        with db.cursor() as c:  
            print("Connexion réussie")  
        db.close()
```

Création de table dans la BD



- ❑ Pour l'exécution d'une requête SQL on utilise la méthode `execute()` fournie par le connecteur MySQL :

```
import mysql.connector
```

```
try:
```

```
    connection_params = {  
        'host': "192.168.1.41",  
        'user': "jerry",  
        'password': "disney",  
        'database': "etudiant"  
    }
```

```
except mysql.connector.Error as e:  
    print("Exception : ", e)
```

```
else:
```

```
    with mysql.connector.connect(**connection_params) as db :  
        with db.cursor() as c:
```

```
            c.execute("CREATE TABLE IF NOT EXISTS personne \  
                (id int NOT NULL AUTO_INCREMENT, nom varchar(25), \  
                 mail varchar(25), age int, PRIMARY KEY (id));")
```

```
            db.commit()
```

```
            db.close()
```

- L'avant dernière ligne du code fait appel à la méthode `commit()`
- La méthode permet de valider les modifications.
- Si la méthode n'est pas appelée, la transaction avec la BD ne sera pas terminée et l'opération SQL n'aura pas son effet.

Création de table dans la BD



- ❑ On peut fournir les valeurs à insérer sous la forme de paramètres à la méthode `execute()` en remplaçant les valeurs par `%s` dans la requête. Les valeurs sont passées en deuxième paramètre.

```
import mysql.connector
try:
    connection_params = {
        'host' : "192.168.1.41",
        'user' : "jerry",
        'password' : "disney",
        'database' : "etudiant"
    }
except mysql.connector.Error as e:
    print("Exception : ", e)
else:
    request = """insert into personne(nom,mail,age) values(%s,%s,%s)"""
    params = ("Tom","tom@disney.com",82)
    with mysql.connector.connect(**connection_params) as db :
        with db.cursor() as c:
            c.execute(request,params)
        db.commit()
        db.close()
```

```
MariaDB [etudiant]> describe personne;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   | PRI | NULL    | auto_increment |
| nom   | varchar(25)   | YES  |     | NULL    |                |
| mail  | varchar(25)   | YES  |     | NULL    |                |
| age   | int(11)       | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.001 sec)

MariaDB [etudiant]> select * from personne;
+----+----+-----+-----+
| id | nom | mail          | age |
+----+----+-----+-----+
| 1  | Tom | tom@disney.com | 82  |
+----+----+-----+-----+
1 row in set (0.000 sec)
```

La méthode `executemany()` permet d'insérer plusieurs lignes en passant un tableau pour les paramètres

Sélection de données



- Pour récupérer des données depuis la BD :
 - il faut passer une requête SQL de type `select` en paramètre de la méthode `execute()` du `curseur`
 - L'appelle la méthode `fetchall()` récupère une liste de n-uplets avec les résultats.

```
request = "select * from personne"
with mysql.connector.connect(**connection_params) as db :
    with db.cursor() as c:
        c.execute(request)
        resultats = c.fetchall()
        for personne in resultats:
            print(personne)
```

```
ad/C:\Users\ivmad\AppData\Local\Programs\Python\Python310\Scripts\Exos/select-mysqlconnect.py
(1, 'Tom', 'tom@disney.com', 82)
(2, 'Jerry', 'tom@disney.com', 82)
(3, 'Pifou', 'pifou@pasglop.fr', 64)
(4, 'Pif', 'pif@vaillant.fr', 65)
(5, 'Mickey', 'mickey@disney.net', 94)
ivmad@macivmad Exos %
```

Sélection de données avec condition



- ❑ La requête **SELECT** peut être paramétrée avec des valeurs d'une clause **where**.
- ❑ Ce mécanisme protège contre une injection SQL car la structure de la requête est fournie quelle que soit la valeur des paramètres.
- ❑ Les paramètres sont spécifiés par **%s** dans la requête et leurs valeurs sont données par un n-uplet passé comme second paramètre à la méthode **execute()** :

```
request = "select id,nom,age from personne where nom = %s"
params = ("Mickey",) # un type list
with mysql.connector.connect(**connection_params) as db :
    with db.cursor() as c:
        c.execute(request, params)
        resultats = c.fetchall()
        for personne in resultats:
            print(personne)
```

A screenshot of a terminal window with a dark background. It shows the output of a SQL query: 'ad/C' on the first line, '(5, 'Mickey', 94)' on the second line, and 'ivmad@macivmad Exos %' on the third line.

```
ad/C
(5, 'Mickey', 94)
ivmad@macivmad Exos %
```



SQL rappel

- La création d'une table avec SQL:

```
CREATE [TEMPORARY] [IF NOT EXISTS]  
TABLE nom_relation (nom_attribut TYPE_ATTRIBUT [OPTIONS]);
```

- **TEMPORARY** donne pour une durée de vie à la table (par exemple: le temps de la connexion de l'utilisateur au serveur, après, elle sera détruite).
 - En l'absence de cette option, la table sera permanente à moins d'être détruite par la commande **DROP TABLE**.
- L'option **IF NOT EXIST** permet de ne créer cette table que si une table de même nom n'existe pas encore.
- La définition de la table se fait à l'intérieur des parenthèses. Il sera listé tous les attributs, clés et indexes de la table.



SQL rappel

- A chacun des enregistrements de la relation il faut associer un identifiant numérique unique de type **entier**.
- Il faut préciser pour cet attribut l'option **NOT NULL** pour le forcer à prendre une valeur de son domaine.
- Il devra aussi être unique, c.à.d. que deux enregistrements ne pourront pas avoir une valeur identique d'identificateur (id).
- Pour indiquer la clé primaire on utilisera l'option **PRIMARY KEY**.
- Facultativement on peut signifier que cette valeur doit s'incrémenter automatiquement à chaque insertion d'un enregistrement. On rajoute alors l'option **AUTO_INCREMENT**.



SQL rappel

- Ajouter un enregistrement:

- Syntaxe d'une "insertion standard" :

`INSERT INTO relation VALUES (liste exhaustive et ordonnée des valeurs);`

- Syntaxe d'une "insertion complète" :

`INSERT INTO relation VALUES
(liste des valeurs),
(liste d'autres valeurs),
(liste d'encore d'autres valeurs), ...`



SQL rappel

- Une clé étrangère est un champ ou un ensemble de champs à l'intérieur d'une table qui se trouve être la clé primaire d'une autre table. Exemple:

```
CREATE TABLE etudiant (id int NOT NULL  
AUTO_INCREMENT, name CHAR(60) NOT NULL, PRIMARY KEY  
(id));
```

```
CREATE TABLE note ( id int NOT NULL AUTO_INCREMENT,  
matiere CHAR(45), note int, owner int NOT NULL,  
PRIMARY KEY (id),  
FOREIGN KEY (owner) REFERENCES etudiant(id) );
```



SQL rappel

- Pour extraire des informations d'une BD.

```
SELECT [ DISTINCT ] attributs  
      [ INTO OUTFILE fichier ]  
      [ FROM relation ]  
      [ WHERE condition ]  
      [ GROUP BY attributs [ ASC | DESC ] ]  
      [ HAVING condition ]  
      [ ORDER BY attributs ]  
      [ LIMIT [a,] b ]
```

- Exemple : la liste des étudiants R&T

```
SELECT nom, prenom FROM Etudiants  
WHERE dut LIKE '%RT%';
```

PYTHON



Exemple

Python & MariaDB





Python - SQLite – BLOB¹

- BLOB signifie *Binary Large Object*.
- SQLite autorise l'utilisation le type de données BLOB pour stocker des données binaires telles que des images, des fichiers vidéo ou toute donnée binaire brute.
- Dans une table une colonne de données peut avoir le type de données BLOB. Par conséquent, on peut y stocker des données binaires.
- En règle générale, un programme Python lit un fichier tel qu'une image et insère son contenu binaire dans une base de données SQLite.
- Pour la démonstration, le programme Python lit les données binaires d'un fichier image, l'insère dans une base de données SQLite, ensuite récupère les données BLOB et écrit les données BLOB sous forme d'image.



Python - SQLite – BLOB²

- Enregistrer l'image dans un champ "blob" (<64Ko) d'une table et reconstruire l'image sous un autre nom.

```
import sqlite3

# Connexion à une BD SQLite
conn = sqlite3.connect('images.db')
cursor = conn.cursor()

# Création de la table
cursor.execute('''CREATE TABLE IF NOT EXISTS documents(
                    id INTEGER PRIMARY KEY,
                    title VARCHAR(255) NOT NULL,
                    data BLOB NOT NULL
                );''')

# Insérer les données binaires
with open('lg_nexus_5.png', 'rb') as file:
    image_data = file.read()
    cursor.execute("INSERT INTO documents (title,data) VALUES (?,?)",\
                    ('Google Nexus 5',image_data,))

# Rechercher uniquement les données binaires
cursor.execute("SELECT data FROM documents WHERE id = 1")
data = cursor.fetchone()[0]

with open('stored_image.jpg', 'wb') as file:
    file.write(data)

# Confirmer les changes et fermeture de la connexion
conn.commit()
conn.close()
```

stored_image.jpg



lg_nexus_5.png





Python - MySQL – Binary - BLOB²

- Enregistrer l'image dans un champ "blob" (<64Ko) binaire

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
import mysql.connector
from mysql.connector import Error
def createTable():
    sql = "create table if not exists images \
        (id int NOT NULL AUTO_INCREMENT PRIMARY KEY, \
         name varchar(25), data mediumblob);"
    return sql
def convertToBinaryData(filename):
    # Convert digital data to binary format
    with open(filename, 'rb') as file:
        binaryData = file.read()
    return binaryData
def insertBLOB(name, photo):
    print("Insérer BLOB dans la table images")
    try:
        # Connexion au serveur MariaDB
        con = mysql.connector.connect(host="localhost",user="ivmad",password="123456",database="test")
        crs = con.cursor()
        # Création de la table images
        sql_create_table_images = createTable()
        result = crs.execute(sql_create_table_images)
        # Insérer les données dans la tables images
        sql_insert_blob_query = """INSERT INTO images (name, data) VALUES (%s,%s)"""
        empPicture = convertToBinaryData(photo)
        # Convertir image en tuple
        insert_blob_tuple = (name, empPicture)
        result = crs.execute(sql_insert_blob_query, insert_blob_tuple)
        con.commit()
        print("Insertion image réussie", result)
    except mysql.connector.Error as error:
        print("Erreur insertion image dans BLOB {}".format(error))
    finally:
        if con.is_connected():
            crs.close()
            con.close()
            print("MySQL connection is closed")
if __name__ == "__main__":
    # Appeler la fonction
    insertBLOB("OnePlus Nord", "/home/ivmad/projects/Smartphones_Python/images/Huawei_Nexus_6P.png")
    insertBLOB("iPhone 12 Pro", "/home/ivmad/projects/Smartphones_Python/images/Lenovo_nexus_6.png")
    insertBLOB("LG Nexus 5", "/home/ivmad/projects/Smartphones_Python/images/lg_nexus_5.png")
    insertBLOB("Google Pixel 3", "/home/ivmad/projects/Smartphones_Python/images/pixel_3.png")
```

- La connexion avec le SGBD est établie avec les identifiants de l'utilisateur
- La table "images" est créée si elle n'existe pas.
- Pour insérer les données dans la table une conversion du binaire est faite.

```
MariaDB [test]> select name from images;
```

```
+-----+
| name          |
+-----+
| OnePlus Nord  |
| iPhone 12 Pro |
| LG Nexus 5    |
| Google Pixel 3|
+-----+
```

```
4 rows in set (0.001 sec)
```

```
MariaDB [test]> █
```




Python - Base64¹

- Base64 est un système de codage binaire en texte représentant des données binaires au format de chaîne ASCII.
- Il est couramment utilisé pour transmettre des données sur Internet ou stocker des données binaires dans un fichier texte ou un champ "BLOB" d'une table de BDR.
- Dans le codage base64, chaque 6 bits des données binaires est représenté comme un caractère parmi un ensemble de 64 caractères.

```
import base64
# Convert the image to base64 format
with open("image.jpg", "rb") as f:
    encoded_image = base64.b64encode(f.read())
```

- Dans l'exemple le fichier image est lu en mode binaire "rb".
- La fonction "b64encode" du module "base64" est appelée pour convertir l'image au format base64.
- L'image encodée est stockée dans la variable "encoded_image".



Python - MySQL – Base64 - BLOB²

- Encoder l'image dans un champ "blob" (<64Ko)

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
import base64
import mysql.connector
from mysql.connector import Error

def createTable():
    sql = "create table if not exists images \
        (id int NOT NULL AUTO_INCREMENT PRIMARY KEY, \
        name varchar(25), data mediumblob);"
    return sql

def convertToBase64(filename):
    # Convert digital data to base64 format
    with open(filename, 'rb') as file:
        encoded_image = base64.b64encode(file.read())
    return encoded_image

def insertBLOB(name, photo):
    print("Insérer BLOB dans la table images")
    try:
        # Connexion au serveur MariaDB
        con = mysql.connector.connect(host="localhost", user="ivmad", password="123456", database="test")
        crs = con.cursor()
        # Création de la table images
        sql_create_table_images = createTable()
        result = crs.execute(sql_create_table_images)
        # Insérer les données dans la table images
        sql_insert_blob_query = """INSERT INTO images (name, data) VALUES (%s,%s)"""
        empPicture = convertToBase64(photo)
        # Convertir image en tuple
        insert_blob_tuple = (name, empPicture)
        result = crs.execute(sql_insert_blob_query, insert_blob_tuple)
        con.commit()
        print("Insertion image réussie", result)
    except mysql.connector.Error as error:
        print("Erreur insertion image dans BLOB {}".format(error))
    finally:
        if (con.is_connected()):
            crs.close()
            con.close()
            print("MySQL connection is closed")

if __name__ == "__main__":
    # Appeler la fonction
    insertBLOB("OnePlus Nord", "/home/ivmad/projects/Smartphones_Python/images/Huawei_Nexus_6P.png")
    insertBLOB("iPhone 12 Pro", "/home/ivmad/projects/Smartphones_Python/images/Lenovo_nexus_6.png")
    insertBLOB("LG Nexus 5", "/home/ivmad/projects/Smartphones_Python/images/lg_nexus_5.png")
    insertBLOB("Google Pixel 3", "/home/ivmad/projects/Smartphones_Python/images/pixel_3.png")
```

- La connexion avec le SGBD est établie avec les identifiants de l'utilisateur
- La table "images" est créée si elle n'existe pas.
- Pour insérer les données dans la table une conversion du binaire est faite avec la méthode base64

select data from images where like "LG Nexus 5"

```
ivmad@debian: ~/projects
-----+
| iVBORw0KGGoAAAANSUHEUgAAAN0AAAGvCAYAAADfWdBkAAABhW
1DQ1BJQ0MgcHJvZmlsZQAAKJF9kT1Iw0AcxV9TpSIVETuIOGSoDm
JRVMRRLq1gEC6Wt0KqDyaVf0KQhSXFfFwLDn4sVh1cnHV1cBUEwQ
8QJ0cnRRcp8X9JoUWMB8f9eHfvcfc0E0plppodE4CqWUYyFhUz2V
Ux8IogQujDOEYlZux1GIanuPrHj6+3kv41ve5P0ePkjMZ4B0J55
huWMQbxD0bls55nzjEipJCfE48ZtAFiR+5Lrv8xrngsMAZQ0Y60U
8cIhYlBsy3MSsaKvE0cVhRNcoXMi4rnLc4q+Uqa96TvzCY01Z5XK
```



Python - MySQL – Base64 - BLOB³

- Extraire image d'un champ base 64 "blob" vers format bitmap

```
#!/usr/bin/python
# -*- coding: utf-8 -*-

import base64
import mysql.connector
from mysql.connector import Error

def write_file(encoded_image, filename):
    # Convertir les données binaires en format bitmap dans un fichier
    with open(filename, 'wb') as file:
        file.write(base64.decodebytes(encoded_image))

def readBLOB(emp_id, photo):
    print("Lecture du BLOB de la table")
    try:
        con = mysql.connector.connect(host="localhost",user="ivmad",password="123456",database="test")
        crs = con.cursor()
        sql_blob_query = """SELECT * from images where id = %s"""
        crs.execute(sql_blob_query, (emp_id,))
        record = crs.fetchall()
        for row in record:
            print("Id = ", row[0], )
            print("Name = ", row[1])
            print("Extraire l'image dans fichier")
            write_file(row[2], photo)
    except mysql.connector.Error as error:
        print("Erreur de lecture données BLOB {}".format(error))
    finally:
        if (con.is_connected()):
            crs.close()
            con.close()
            print("MySQL connection is closed")

if __name__ == "__main__":
    # Appeler la fonction avec paramètres
    readBLOB(3, "/home/ivmad/projects/Smartphones_Python/images/google_pixel_3.png")
```



Pour décoder l'image du BLOB
on utilise la fonction :

```
import base64 with
open("imageToSave.png","wb") as fh:
fh.write(base64.decodebytes(img_data))
```